

Deep Learning for Monkeypox and Skin Disease Classification

The complete repository is available online: • OneDrive:

https://1drv.ms/f/s!AnsUjivQBPA7hOdfT4D_w8925r1pOg?e=WbmH80

The Dataset from kaggle :

<https://www.kaggle.com/datasets/dipuiucse/monkeypoxskinimagedataset>

Setup and Data Preparation

```
In [ ]: import os
os.environ['KAGGLE_USERNAME'] = 'your_username'
os.environ['KAGGLE_KEY'] = 'your_key'
```

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
```

```
In [ ]: !kaggle datasets download dipuiucse/monkeypoxskinimagedataset
```

```
In [ ]: !unzip -q "/content/monkeypoxskinimagedataset.zip"
```

```
In [ ]: train_dir = "/content/Monkeypox Skin Image Dataset"
```

```
In [ ]: import os
print(os.listdir(train_dir))
```

```
In [ ]: !ls "/content/Monkeypox Skin Image Dataset"
```

```
In [ ]: # Install necessary packages
!pip install gradio
```

Library Imports and Environment Setup

```
In [ ]: # -----
# Core Python Libraries
# -----
import os
import random
import glob
import hashlib

# -----
```

```

# Data Handling & Computation
# -----
import numpy as np
import pandas as pd

# -----
# Visualization
# -----
import matplotlib.pyplot as plt
import seaborn as sns

# -----
# Image Processing
# -----
from PIL import Image
import cv2

# -----
# Scikit-Learn Utilities
# -----
from sklearn.model_selection import train_test_split, KFold, StratifiedKFold
from sklearn.preprocessing import LabelEncoder
from sklearn.utils.class_weight import compute_class_weight
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    roc_curve,
    auc,
    precision_recall_curve
)

# -----
# TensorFlow / Keras Core
# -----
import tensorflow as tf
from tensorflow.keras.models import Model, load_model
from tensorflow.keras import Input
from tensorflow.keras.layers import (
    Dense,
    Dropout,
    GlobalAveragePooling2D,
    Average,
    Input
)
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.preprocessing.image import load_img, img_to_array
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import (
    EarlyStopping,
    ReduceLROnPlateau,
    ModelCheckpoint
)

# -----
# Keras Pretrained Applications
# -----

```

```

from tensorflow.keras.applications import (
    densenet,
    resnet,
    resnet_v2,
    inception_resnet_v2,
    mobilenet_v2,
    nasnet,
    efficientnet,
    xception,
    vgg16,
    vgg19,
    inception_v3
)
from tensorflow.keras.applications.densenet import preprocess_input as densenet_pre

# -----
# Gradio Web Interface
# -----
import gradio as gr

# -----
# SciPy
# -----
from scipy import interpolate

# -----
# seeds for reproducibility
# -----
random.seed(42)
np.random.seed(42)
tf.random.set_seed(42)

```

Identify duplicate images using hashing

```

In [ ]: # Initialize storage
hash_map = {}      # To store hash : original file path
duplicates = []    # To track duplicate file paths

# Traverse all images in subfolders
all_files = glob.glob("/content/Monkeypox Skin Image Dataset/*/*")

for file_path in all_files:
    with open(file_path, "rb") as f:
        file_content = f.read()
        file_hash = hashlib.md5(file_content).hexdigest() # Unique identifier

    if file_hash in hash_map:
        # Duplicate found
        original_path = hash_map[file_hash]
        duplicates.append((file_path, original_path))
    else:
        hash_map[file_hash] = file_path # First time this content is seen

```

```

# Print and delete duplicates
print(f"\nTotal duplicate pairs found: {len(duplicates)}")

for idx, (dup_path, original_path) in enumerate(duplicates, start=1):
    print(f"Duplicate #{idx}:")
    print(f"    Deleting duplicate: {dup_path}")
    print(f"    Keeping original: {original_path}")
    print("-" * 50)
    # Display images side by side

    dup_img = Image.open(dup_path)
    orig_img = Image.open(original_path)

    fig, axes = plt.subplots(1, 2, figsize=(8, 4))
    axes[0].imshow(dup_img)
    axes[0].set_title("Duplicate")
    axes[0].axis("off")
    axes[1].imshow(orig_img)
    axes[1].set_title("Original")
    axes[1].axis("off")
    plt.show()
    try:
        os.remove(dup_path)
        print("Deleted successfully.")
    except Exception as e:
        print(f"Error deleting file: {e}")

```

Create a DataFrame from images directory

```

In [ ]: def create_image_dataframe(base_dir):
        """
        Walks through a dataset directory where each subfolder is a class,
        and builds a DataFrame listing each image file and its label.

        Parameters:
        - base_dir (str): The path to the main dataset folder.

        Returns:
        - DataFrame with columns: 'filepath' and 'label'.
        """

        # Initialize an empty list to store image info
        data_list = []

        # Loop over each class folder in the main directory
        for class_name in sorted(os.listdir(base_dir)):
            class_path = os.path.join(base_dir, class_name)

            # Make sure it's a folder (not a file)
            if os.path.isdir(class_path):

                # Loop through all files in the class folder
                for fname in os.listdir(class_path):

```

```

        # Only include image files (jpg, png, jpeg)
        if fname.lower().endswith(('.png', '.jpg', '.jpeg')):
            full_path = os.path.join(class_path, fname)

        # Add a tuple of (filepath, label) to the list
        data_list.append((full_path, class_name))

    # Convert the list into a DataFrame
    df = pd.DataFrame(data_list, columns=['filepath', 'label'])
    return df

```

```

In [ ]: # Define your dataset path
train_dir = "/content/Monkeypox Skin Image Dataset"

# Create the DataFrame
df_all = create_image_dataframe(train_dir)

# Print results
print("Total images found:", len(df_all))
print(df_all.head())

```

Label Encoding and Train-Test Split

Using `stratify=df_all['label_idx']` ensures that both the training+validation and test sets contain the same proportion of each class. This avoids imbalanced testing, which could bias model evaluation.

```

In [ ]: # -----
# 3. Encode Labels
# -----

label_encoder = LabelEncoder()
df_all['label_idx'] = label_encoder.fit_transform(df_all['label'])
num_classes = df_all['label_idx'].nunique()
print("Unique Classes:", label_encoder.classes_)

# -----
# Test Set Split
# -----

# Split the full dataset into training+validation and test sets (80-20 split)
df_train_val, df_test = train_test_split(
    df_all,
    test_size=0.2,                # Reserve 20% for testing
    stratify=df_all['label_idx'], # Ensure class balance in both sets
    random_state=42               # Makes the split reproducible
)

print("Training+Validation Set:", len(df_train_val))
print("Test Set:", len(df_test))

```

Data Generators and Preprocessing

```

In [ ]: # -----
# Data Generators using flow_from_dataframe
# -----
def get_preprocessing_function(model_name):
    """
    Returns the appropriate preprocess_input function based on the model name.
    Ensures images are preprocessed consistently with the given architecture.

    Args:
        model_name (str): Name of the model architecture.

    Returns:
        function: Corresponding preprocess_input function.

    Raises:
        ValueError: If no matching model name is found.
    """

    model_name_lower = model_name.lower()

    # DenseNet family
    if "densenet" in model_name_lower:
        return densenet.preprocess_input

    # ResNet V2 variants
    elif "resnet50v2" in model_name_lower or "resnet101v2" in model_name_lower or "
        return resnet_v2.preprocess_input

    # ResNet (original) variants
    elif "resnet50" in model_name_lower or "resnet101" in model_name_lower or "resn
        return resnet.preprocess_input

    # Inception-ResNet-V2
    elif "inceptionresnetv2" in model_name_lower:
        return inception_resnet_v2.preprocess_input

    # MobileNetV2
    elif "mobilenetv2" in model_name_lower:
        return mobilenet_v2.preprocess_input

    # NASNet
    elif "nasnetlarge" in model_name_lower or "nasnetmobile" in model_name_lower:
        return nasnet.preprocess_input

    # EfficientNet (B0-B7)
    elif "efficientnet" in model_name_lower:
        return efficientnet.preprocess_input

    # Xception
    elif "xception" in model_name_lower:
        return xception.preprocess_input

    # VGG family
    elif "vgg16" in model_name_lower:
        return vgg16.preprocess_input

```

```

elif "vgg19" in model_name_lower:
    return vgg19.preprocess_input

# InceptionV3
elif "inceptionv3" in model_name_lower:
    return inception_v3.preprocess_input

else:
    raise ValueError(f"No matching preprocessing found for '{model_name}'")

def get_data_generators_from_df(df_train, df_val, model_name, image_size=(224, 224)
    """
    Creates Keras image data generators for training and validation sets.

    Args:
        df_train (pd.DataFrame): DataFrame containing training image paths and labels
        df_val (pd.DataFrame): DataFrame containing validation image paths and labels
        model_name (str): Name of the model to select the appropriate preprocessing
        image_size (tuple, optional): Target size for images. Defaults to (224, 224)
        batch_size (int, optional): Number of samples per batch. Defaults to 32.

    Returns:
        tuple: (train_generator, val_generator) - Keras data generators.
    """

    #save_dir = "/content/Alex"
    #if not os.path.exists(save_dir):
    #    os.makedirs(save_dir)

    # Dynamically choose the correct preprocess function
    preprocess_func = get_preprocessing_function(model_name)

    train_datagen = ImageDataGenerator(
        preprocessing_function=preprocess_func,
        rotation_range=30,
        zoom_range=0.3,
        width_shift_range=0.2,
        height_shift_range=0.2,
        horizontal_flip=True,
        vertical_flip=True
    )
    val_datagen = ImageDataGenerator(preprocessing_function=preprocess_func)

    train_generator = train_datagen.flow_from_dataframe(
        dataframe=df_train,
        x_col='filepath',
        y_col='label',
        target_size=image_size,
        class_mode='categorical',
        batch_size=batch_size,
        shuffle=True
        #save_to_dir=save_dir, # New: Directory for saved images
        #save_prefix='aug', # Prefix for the filenames
        #save_format='png' # Format of the saved images#
    )

```

```

val_generator = val_datagen.flow_from_dataframe(
    dataframe=df_val,
    x_col='filepath',
    y_col='label',
    target_size=image_size,
    class_mode='categorical',
    batch_size=batch_size,
    shuffle=False
)

return train_generator, val_generator

```

Model Utilities

```

In [ ]: # -----
# Model Utilities (Head, Unfreeze)
# -----
"""Constructs a classification head on top of a pre-trained CNN base model.

Args:
    base_model (keras.Model): The convolutional base model (without top layers)
    num_classes (int): Number of output classes.

Returns:
    keras.layers.Layer: The final output layer for classification."""

def build_model_head(base_model, num_classes):

    x = base_model.output
    x = GlobalAveragePooling2D()(x)
    x = Dense(256, activation='relu')(x)
    x = Dropout(0.2)(x)
    output = Dense(num_classes, activation='softmax')(x)
    return output

"""Unfreezes a specified percentage of the last layers in a model for fine-tuning.

Args:
    model (keras.Model): The model whose layers are to be unfrozen.
    percentage (float): Proportion of the model's layers to unfreeze, from the
                        Value must be between 0 and 1.

Returns:
    keras.Model: The modified model with selective layers set as trainable."""

def unfreeze_layers(model, percentage=0.5):
    # Freeze all layers first
    for layer in model.layers:
        layer.trainable = False
    # Then unfreeze the last 'percentage' of layers
    total_layers = len(model.layers)

```



```

trainable_layers = int(percentage * total_layers)
for layer in model.layers[-trainable_layers:]:
    layer.trainable = True
return model

```

Model Selection

```

In [ ]: # -----
# Models Dictionary
# -----
"""This dictionary maps model names to their corresponding pre-trained base archite

models_dict = {

    'MobileNetV2': tf.keras.applications.MobileNetV2(weights='imagenet', include_top
    'Xception': xception.Xception(weights='imagenet', include_top=False, input_shap
    'InceptionV3': inception_v3.InceptionV3(weights='imagenet', include_top=False,
    'EfficientNetB7': tf.keras.applications.EfficientNetB7(weights='imagenet', incl
    'DenseNet169': densenet.DenseNet169(weights='imagenet', include_top=False, input
    'ResNet50': resnet.ResNet50(weights='imagenet', include_top=False, input_shape
    'VGG16' : vgg16.VGG16(weights='imagenet', include_top=False, input_shape=(224,
}

```

K-Fold Cross-Validation Loop

```

In [ ]: # -----
# 7. K-Fold Cross-Validation Setup
# -----
FOLDS = 5
Skf = StratifiedKFold(n_splits=FOLDS, shuffle=True, random_state=42)
performance_dfs = [] # List to store results from each fold
fold_idx = 1

# K-Fold Cross-Validation Loop
for train_idx, val_idx in Skf.split(df_all['filepath'], df_all['label_idx']):
    print(f"\n== K-Fold {fold_idx}/{FOLDS} ==")
    print("Train:", len(train_idx), "Validation:", len(val_idx))
    # Inside the K-fold Loop:

    # Create train and validation DataFrames for this fold
    df_train = df_all.iloc[train_idx].reset_index(drop=True)
    df_val = df_all.iloc[val_idx].reset_index(drop=True)

    # Compute class weights for this fold using numeric labels
    train_labels = df_train['label_idx'].values
    unique_classes = np.unique(train_labels)
    cw_array = compute_class_weight('balanced', classes=unique_classes, y=train_labels)
    class_weights = dict(enumerate(cw_array))

```

```

print("Fold class weights:", class_weights)

# Dictionary to store metrics for each model in this fold
fold_results = {'Fold': [], 'Model': [], 'Accuracy': [], 'Precision': [], 'Recall': []}

# Loop through each model architecture want to test
for model_name, base_model in models_dict.items():

    print(f"\n--- Training {model_name} in Fold {fold_idx} ---")

    # Define callbacks for early stopping and model checkpointing
    callback_list = [
        EarlyStopping(
            monitor='val_loss',
            patience=3,
            restore_best_weights=True
        ),
        ReduceLROnPlateau(
            monitor='val_loss',
            factor=0.1,
            patience=2,
            min_lr=1e-7
        ),
        ModelCheckpoint(
            filepath=f'{model_name}_fold{fold_idx}_best.h5',
            monitor='val_loss',
            save_best_only=True,
            verbose=1
        )
    ]

    # Create generators from the DataFrame splits
    train_gen, val_gen = get_data_generators_from_df(df_train, df_val, model_name)

    # Build the full model (base + head)

    predictions = build_model_head(base_model, num_classes)
    model = Model(inputs=base_model.input, outputs=predictions)

    # Stage 1: Freeze base model layers and train the head
    for layer in base_model.layers:
        layer.trainable = False

    model.compile(optimizer=Adam(1e-4),
                  loss='categorical_crossentropy',
                  metrics=['accuracy', tf.keras.metrics.Precision(), tf.keras.metrics.Recall()])

    history1 = model.fit(train_gen,
                        epochs=20,
                        validation_data=val_gen,
                        class_weight=class_weights,
                        callbacks=callback_list )

```

```

# Stage 2: Unfreeze all layers and fine-tune
model = unfreeze_layers(model, percentage=0.8)
model.compile(optimizer=Adam(1e-5),
              loss='categorical_crossentropy',
              metrics=['accuracy', tf.keras.metrics.Precision(), tf.keras.m

history2 = model.fit(train_gen,
                    epochs=10,
                    validation_data=val_gen,
                    class_weight=class_weights,
                    callbacks=callback_list )

# Evaluate the model on the validation set
val_preds = model.predict(val_gen)
y_pred = np.argmax(val_preds, axis=1)
# Use the generator's classes attribute as numeric labels
y_true = val_gen.classes

# Generate classification report
report = classification_report(y_true, y_pred, output_dict=True)
acc = report['accuracy']
prec = report['weighted avg']['precision']
rec = report['weighted avg']['recall']
f1 = report['weighted avg']['f1-score']

fold_results['Fold'].append(fold_idx)
fold_results['Model'].append(model_name)
fold_results['Accuracy'].append(acc)
fold_results['Precision'].append(prec)
fold_results['Recall'].append(rec)
fold_results['F1-Score'].append(f1)

# confusion matrix for this model and fold
cm = confusion_matrix(y_true, y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
           xticklabels=label_encoder.classes_,
           yticklabels=label_encoder.classes_)
plt.title(f"{model_name} - Fold {fold_idx} Confusion Matrix")
plt.savefig(f"{model_name}_Fold_{fold_idx}_Confusion_Matrix.png")
plt.show()

# Save all fold results
performance_dfs.append(pd.DataFrame(fold_results))
fold_idx += 1

```

Plot Training Metric & Test Set Evaluation

```

In [ ]: # Combine accuracy from history1 and history2
combined_accuracy = history1.history['accuracy'] + history2.history['accuracy']

```

```

combined_val_accuracy = history1.history['val_accuracy'] + history2.history['val_ac

plt.figure(figsize=(8,6))
plt.plot(combined_accuracy, label='Train')
plt.plot(combined_val_accuracy, label='Validation')
plt.title(f'{model_name} Model Accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(loc='upper left')
plt.show()

# for Loss
combined_loss = history1.history['loss'] + history2.history['loss']
combined_val_loss = history1.history['val_loss'] + history2.history['val_loss']

plt.figure(figsize=(8,6))
plt.plot(combined_loss, label='Train')
plt.plot(combined_val_loss, label='Validation')
plt.title(f'{model_name} Model Loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(loc='upper left')
plt.show()

```

```

In [ ]: FOLDS = 5
fold_idx = 1
print(f"\n== K-Fold {fold_idx}/{FOLDS} ==")
print("Train:", len(train_idx), "Validation:", len(val_idx))

fold_idx = 1
print(f"\nFold {fold_idx} Class Distribution:")
print("Training Set:")
print(df_train['label'].value_counts())
print("\nValidation Set:")
print(df_val['label'].value_counts())

plt.figure(figsize=(12, 5))

# Training Distribution
plt.subplot(1, 2, 1)
sns.countplot(data=df_all, x='label', order=df_all['label'].value_counts().index)
plt.title('Overall Class Distribution')
plt.xticks(rotation=45)

# Example Validation Distribution (for 1 fold)
plt.subplot(1, 2, 2)
sns.countplot(data=df_val, x='label', order=df_all['label'].value_counts().index)
plt.title('Example Validation Fold Distribution')
plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

# -----
# 8. Final Summary Plot Results
# -----
final_df = pd.concat(performance_dfs, ignore_index=True)

```

```

print("\nAll Fold Results:")
print(final_df)

avg_df = final_df.groupby('Model').mean(numeric_only=True)
print("\nAverage Results by Model:")
print(avg_df)

avg_df[['Accuracy', 'Precision', 'Recall', 'F1-Score']].plot(kind='bar', figsize=(10,6)
plt.ylim(0, 1)
plt.title("Average CV Performance")
plt.ylabel("Score")
plt.show()

```

```

In [ ]: # Save cross-validation results to CSV
final_df.to_csv('cross_validation_results.csv', index=False)
print("Cross-validation results saved to cross_validation_results.csv")

# Save average performance results to CSV
avg_df.to_csv('average_performance_results.csv')
print("Average performance results saved to average_performance_results.csv")

```

```

In [ ]: model.summary()

```

```

In [ ]: # Test Set Evaluation
print("\n=== Comprehensive Test Set Evaluation ===")
# the model to evaluate
model_names = ['DenseNet169', 'ResNet50', 'EfficientNetB7', 'InceptionV3', 'VGG16', 'VGG
FOLDS = 5

# Dictionary to store results for each model
test_results = {}
test_predictions = {}
test_probabilities = {}

```

```

In [ ]: # Test Data Generator Creation Function
def create_test_generator(model_name):
    """Create a test data generator with appropriate preprocessing for the model"""
    test_preprocess = get_preprocessing_function(model_name)
    test_datagen = ImageDataGenerator(preprocessing_function=test_preprocess)

    return test_datagen.flow_from_dataframe(
        dataframe=df_test,
        x_col='filepath',
        y_col='label',
        target_size=(224, 224),
        class_mode='categorical',
        batch_size=32,
        shuffle=False
    )

```

```

In [ ]: from sklearn.metrics import classification_report, confusion_matrix

def save_confusion_matrix(y_true, predictions, class_names, title, filename):
    """Create and save a confusion matrix visualization"""
    cm = confusion_matrix(y_true, predictions)

```



```

plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.title(title)
plt.legend(loc="lower right" if curve_type == 'roc' else "lower left")

plt.tight_layout()
plt.savefig(filename)
plt.close()

```

```

In [ ]: def plot_fold_roc_curves(model_name, fold_probabilities, fold_predictions, y_true,
    """
    Plot ROC curves for each fold of a model

    Parameters:
    - model_name: Name of the model
    - fold_probabilities: List of probability arrays for each fold
    - fold_predictions: List of prediction arrays for each fold
    - y_true: Ground truth labels
    - class_idx: If specified, plot ROC for this specific class, otherwise use over
    """
    plt.figure(figsize=(10, 8))

    # Setup the plot
    plt.plot([0, 1], [0, 1], 'k--', lw=2)
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel('False Positive Rate')
    plt.ylabel('True Positive Rate')
    plt.title(f'Area Under Curve')

    # Define a list of line styles and colors for distinction
    colors = ['purple', 'red', 'blue', 'green', 'yellow', 'cyan', 'magenta', 'brown']
    line_styles = ['-', '--', '-.', ':']

    # Plot each fold
    for fold_idx, probs in enumerate(fold_probabilities):
        # Get the probabilities for this fold
        if class_idx is not None:
            # For a specific class
            y_score = probs[:, class_idx]
            y_true_binary = np.zeros_like(y_true)
            y_true_binary[y_true == class_idx] = 1

        else:
            # For overall multi-class ROC

            y_score = probs.max(axis=1)
            predictions = fold_predictions[fold_idx]
            y_true_binary = (y_true == predictions).astype(int)

        # Calculate ROC curve with many threshold points for smoother appearance
        fpr, tpr, _ = roc_curve(y_true_binary, y_score, drop_intermediate=False)
        roc_auc = auc(fpr, tpr)

```

```

# Plot with distinct color and style
color_idx = fold_idx % len(colors)
style_idx = (fold_idx // len(colors)) % len(line_styles)
plt.plot(fpr, tpr,
         color=colors[color_idx],
         linestyle=line_styles[style_idx],
         lw=2,
         label=f'Fold {fold_idx+1} - {roc_auc:.4f}')

plt.legend(loc="lower right")
plt.grid(True)

# Save with model name and class info
class_suffix = f"_class{class_idx}" if class_idx is not None else ""
plt.savefig(f'{model_name}_fold_roc_curves{class_suffix}.png')
plt.close()

```

```

In [ ]: # 1. Evaluate each individual model
print("\n=== Individual Model Evaluation ===")

# Get the ground truth labels
temp_generator = create_test_generator(model_names[0])
y_true = np.array(temp_generator.classes)
class_names = list(temp_generator.class_indices.keys())
print(f"Test set size: {len(y_true)} samples")
print(f"Class distribution: {np.bincount(y_true)}")

for model_name in model_names:
    print(f"\nEvaluating {model_name} on test set...")
    model_files = glob.glob(f"{model_name}_fold*_best.h5")

    if not model_files:
        print(f"No saved models found for {model_name}.")
        continue

    fold_probabilities = []
    fold_predictions = []

    # Initialize metrics for each fold
    fold_metrics = {}
    fold_metrics['accuracy'] = []
    fold_metrics['precision'] = []
    fold_metrics['recall'] = []
    fold_metrics['f1'] = []

    # Evaluate each fold's model
    for fold_idx, model_path in enumerate(model_files, 1):
        print(f"    Evaluating fold {fold_idx} model: {model_path}")
        test_generator = create_test_generator(model_name)

        # Load model and evaluate
        model = tf.keras.models.load_model(model_path)
        metrics = model.evaluate(test_generator, verbose=0)

        # Store fold metrics
        fold_metrics['accuracy'].append(metrics[1])

```



```

fold_metrics['precision'].append(metrics[2])
fold_metrics['recall'].append(metrics[3])
f1 = 2 * (metrics[2] * metrics[3]) / (metrics[2] + metrics[3] + 1e-10)
fold_metrics['f1'].append(f1)

# Get predictions
test_generator.reset()
predictions = model.predict(test_generator, verbose=0)
fold_probabilities.append(predictions)
fold_predictions.append(np.argmax(predictions, axis=1))

# Process results if we have any folds
if fold_probabilities:
    # Create ROC curves for each fold

    plot_fold_roc_curves(model_name, fold_probabilities, fold_predictions, y_tr

# create class-specific ROC curves if there are multiple classes
if len(class_names) > 1:
    for class_idx, class_name in enumerate(class_names):
        plot_fold_roc_curves(
            f"{model_name}_{class_name}",
            fold_probabilities,
            fold_predictions,
            y_true,
            class_idx=class_idx
        )

# Average predictions across folds
avg_probabilities = np.mean(np.array(fold_probabilities), axis=0)
avg_predictions = np.argmax(avg_probabilities, axis=1)

# Store results
test_predictions[model_name] = avg_predictions
test_probabilities[model_name] = avg_probabilities
test_results[model_name] = {}
test_results[model_name]['accuracy'] = np.mean(fold_metrics['accuracy'])
test_results[model_name]['precision'] = np.mean(fold_metrics['precision'])
test_results[model_name]['recall'] = np.mean(fold_metrics['recall'])
test_results[model_name]['f1'] = np.mean(fold_metrics['f1'])

# Print results
print("\n" + model_name + " average performance across " + str(len(model_fi

for metric, value in test_results[model_name].items():
    metric_name = metric.capitalize()
    formatted_value = round(value, 4)
    print(" " + metric_name + ": " + str(formatted_value))

# visualizations
save_confusion_matrix(
    y_true, avg_predictions, class_names,
    f'{model_name} Confusion Matrix',
    f'{model_name}_test_confusion_matrix.png'
)

```

```

save_classification_report(
    y_true, avg_predictions, class_names,
    f'{model_name}_test_classification_report.csv'
)

```

Ensemble Model Creation and Evaluation

```

In [ ]: # 2. Create and Evaluate the Ensemble Model
print("\n== Creating and Evaluating Ensemble Model ==")

model_names = ['DenseNet169', 'ResNet50']
FOLDS = 5

# Load all models from different folds for ensemble creation
loaded_models = []

for model_name in model_names:
    for fold in range(1, FOLDS + 1):
        file_pattern = f'{model_name}_fold{fold}_best.h5'
        model_files = glob.glob(file_pattern)

        if model_files:
            saved_model_path = model_files[0]
            print(f"Loading {model_name} from fold {fold}: {saved_model_path}")
            model = tf.keras.models.load_model(saved_model_path)
            loaded_models.append(model)
        else:
            print(f"No saved model found for {model_name} in fold {fold}.")

if not loaded_models:
    print("No models were loaded for ensemble creation. Skipping ensemble model.")
else:
    # Create a test generator for evaluation
    test_generator = create_test_generator(model_names[0]) # Use preprocessing fro

# Using predictions instead of connecting models directly ==
# This is simpler and more reliable, but slightly less efficient

# First, prepare the test data
test_generator.reset()
test_data = []
test_labels = []

# Collect a batch of data for prediction
for i in range(len(test_generator)):
    x, y = test_generator[i]
    test_data.append(x)
    test_labels.append(y)

test_data = np.vstack(test_data)
test_labels = np.vstack(test_labels)
y_true = np.argmax(test_labels, axis=1)

```

```

# Get predictions from each model separately
all_predictions = []
for i, model in enumerate(loaded_models):
    print(f"Getting predictions from model {i+1}/{len(loaded_models)}")
    predictions = model.predict(test_data, verbose=0)
    all_predictions.append(predictions)

# Average the predictions (soft voting)
ensemble_probabilities = np.mean(all_predictions, axis=0)
ensemble_predictions = np.argmax(ensemble_probabilities, axis=1)

# Calculate and store ensemble metrics
ensemble_report = save_classification_report(
    y_true, ensemble_predictions, class_names,
    'ensemble_test_classification_report.csv'
)

test_results['Ensemble'] = calculate_metrics(ensemble_predictions, y_true, clas
test_probabilities['Ensemble'] = ensemble_probabilities

# Print ensemble performance
print("\nEnsemble model performance:")
for metric, value in test_results['Ensemble'].items():
    print(f" {metric.capitalize()}: {value:.4f}")

# Generate ensemble visualizations
save_confusion_matrix(
    y_true, ensemble_predictions, class_names,
    'Ensemble Model Confusion Matrix (Test Set)',
    'ensemble_test_confusion_matrix.png'
)

# Define input layer
ensemble_input = Input(shape=(224, 224, 3))

# Run each loaded model on the same input
model_outputs = [model(ensemble_input) for model in loaded_models]

# Average their outputs (soft voting)
averaged_output = Average()(model_outputs)

# Define the final ensemble model
ensemble_model = Model(inputs=ensemble_input, outputs=averaged_output)

# Save for future use
ensemble_model.save("ensemble_model.keras", save_format="keras")
print("Ensemble model saved and defined.")

```

```

In [ ]: # 3. comparison visualizations
if test_results:
    models = list(test_results.keys())
    metrics = ['accuracy', 'precision', 'recall', 'f1']

    # Create grouped bar chart

```

```

x = np.arange(len(models))
width = 0.2
fig, ax = plt.subplots(figsize=(14, 8))

for i, metric in enumerate(metrics):
    values = [test_results[model][metric] for model in models]
    bars = ax.bar(x + i*width, values, width, label=metric.capitalize())

    # Add value labels on bars
    for j, bar in enumerate(bars):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.01,
                f'{values[j]:.3f}', ha='center', va='bottom', fontsize=8, rotati

ax.set_ylabel('Score')
ax.set_title('Model Performance Comparison on Test Set')
ax.set_xticks(x + width * 1.5)
ax.set_xticklabels(models)
ax.legend(loc='best')
ax.set_ylim(0, 1.0)

plt.tight_layout()
plt.savefig('model_comparison_test_set.png')
plt.close()

# 4. ROC and precision-recall curves
if test_probabilities:
    # ROC curves
    plot_curves_by_class(class_names, test_probabilities, y_true, curve_type='roc')

    # Precision-recall curves
    plot_curves_by_class(class_names, test_probabilities, y_true, curve_type='pr')

# 5. summary table with all results
if test_results:
    summary_rows = []

    for model_name, metrics in test_results.items():
        row = {'Model': model_name}
        row.update(metrics)

        # Add per-class precision, recall, f1
        if model_name == 'Ensemble':
            report = ensemble_report
        else:
            report = classification_report(y_true, test_predictions[model_name],
                                          target_names=class_names, output_dict=True)

        for class_name in class_names:
            row[f'{class_name}_precision'] = report[class_name]['precision']
            row[f'{class_name}_recall'] = report[class_name]['recall']
            row[f'{class_name}_f1'] = report[class_name]['f1-score']

        summary_rows.append(row)

# save summary DataFrame
summary_df = pd.DataFrame(summary_rows)

```

```
summary_df.to_csv('test_set_evaluation_summary.csv', index=False)
print("\nTest set evaluation complete. Results saved to CSV files and images.")
```

```
In [ ]: ensemble_model.summary()
```

```
In [ ]: model.summary()
```

Gradio Interface

```
In [ ]: import tensorflow as tf
import numpy as np
from tensorflow.keras.preprocessing import image
from tensorflow.keras.applications.densenet import preprocess_input as densenet_pre
import gradio as gr

# Load the ensemble model
ensemble_model = tf.keras.models.load_model("/content/ensemble_model.keras")

# Class labels and input size
CLASS_LABELS = ["Chickenpox", "Measles", "Monkeypox", "Normal"]
IMG_SIZE = (224, 224)

def predict_image(uploaded_img):
    # Resize and preprocess image
    resized_img = uploaded_img.resize(IMG_SIZE)
    img_array = image.img_to_array(resized_img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array = densenet_preprocess(img_array)

    # Predict using the loaded ensemble model
    predictions = ensemble_model.predict(img_array)
    pred_index = np.argmax(predictions, axis=1)[0]

    return f"Predicted Class: {CLASS_LABELS[pred_index]}"

# Build and launch the Gradio interface
interface = gr.Interface(
    fn=predict_image,
    inputs=gr.Image(type="pil"),
    outputs="text",
    title="Monkeypox Skin Image Classifier (Ensemble)",
    description="Upload a skin image. The deep ensemble model (DenseNet169 + ResNet
                "into one of four categories: Monkeypox, Chickenpox, Measles, or No
    )

interface.launch()
```

Grad-CAM Visualization

```
In [ ]: def gradcam_single_model(model, img_array, target_layer):
        """
        Computes a Grad-CAM heatmap for a given model and input image.

        Args:
            model: A tf.keras.Model object.
            img_array: A preprocessed image array with shape (1, H, W, 3).
            target_layer: Name of the model's last convolutional layer.

        Returns:
            heatmap: A 2D numpy array normalized to [0, 1], representing class-discrimi
        """
        # get feature maps and predictions
        grad_model = tf.keras.models.Model([model.input],
                                           [model.get_layer(target_layer).output, model

        with tf.GradientTape() as tape:
            conv_outputs, predictions = grad_model(img_array)
            class_index = tf.argmax(predictions[0])
            loss = predictions[:, class_index]

            # Compute gradients and their global average
            grads = tape.gradient(loss, conv_outputs)[0]
            weights = tf.reduce_mean(grads, axis=(0, 1))

            # Compute weighted sum of conv feature maps
            cam = tf.reduce_sum(tf.multiply(conv_outputs[0], weights), axis=-1)

            # Normalize the heatmap
            heatmap = np.maximum(cam, 0)
            heatmap = heatmap / tf.math.reduce_max(heatmap + 1e-8)

        return heatmap.numpy()
```

```
In [ ]: def gradcam_ensemble(models, img_array, layer_names):
        """
        Computes an average Grad-CAM heatmap from multiple models.

        Args:
            models: A list of tf.keras.Model objects.
            img_array: A single preprocessed image (1, height, width, 3).
            layer_names: A list of conv-layer names corresponding to the models.
        """
        heatmaps = []
        for model, layer_name in zip(models, layer_names):
            hm = gradcam_single_model(model, img_array, layer_name)
            heatmaps.append(hm)

        # Average the heatmaps
        avg_heatmap = np.mean(np.stack(heatmaps), axis=0)

        # Normalize again
        avg_heatmap = np.maximum(avg_heatmap, 0)
        if avg_heatmap.max() != 0:
            avg_heatmap /= avg_heatmap.max()
```

```
return avg_heatmap
```

```
In [ ]: # Example usage -----
ResNet50 = tf.keras.models.load_model("/content/ResNet50_fold5_best.h5")
DenseNet169 = tf.keras.models.load_model("/content/DenseNet169_fold5_best.h5")
models = [ResNet50,DenseNet169] # If more models, add them here
layer_names = [ "conv5_block3_3_conv","conv5_block16_2_conv"] # Change if your mod

image_paths = [
    "/content/Monkeypox Skin Image Dataset/Chickenpox/chickenpox18.png",
    "/content/Monkeypox Skin Image Dataset/Measles/measles14.png",
    "/content/Monkeypox Skin Image Dataset/Monkeypox/monkeypox101.png",
    "/content/Monkeypox Skin Image Dataset/Normal/normal115.png"
]

for img_path in image_paths:
    img = load_img(img_path, target_size=(224, 224))
    arr = img_to_array(img)
    arr = np.expand_dims(arr, axis=0)

    # Generate ensemble Grad-CAM heatmap
    heatmap = gradcam_ensemble(models, arr, layer_names)

    # Overlay heatmap on original image
    hm_resized = cv2.resize(heatmap, (224, 224))
    hm_color = cv2.applyColorMap(np.uint8(255 * hm_resized), cv2.COLORMAP_JET)
    img_bgr = cv2.cvtColor(np.array(img), cv2.COLOR_RGB2BGR)
    overlay = cv2.addWeighted(img_bgr, 0.6, hm_color, 0.4, 0)

    # Display
    plt.figure(figsize=(12,4))
    plt.subplot(1,3,1)
    plt.imshow(img)
    plt.axis("off")
    plt.title("Original")
    plt.subplot(1,3,2)
    plt.imshow(heatmap, cmap="jet")
    plt.axis("off")
    plt.title("Heatmap")
    plt.subplot(1,3,3)
    plt.imshow(cv2.cvtColor(overlay, cv2.COLOR_BGR2RGB))
    plt.axis("off")
    plt.title("Overlay")
    plt.suptitle(img_path)
    plt.show()
```

Final Performance Summary Plot

```
In [ ]: # Read the CSV file
df = pd.read_csv('/content/test_set_evaluation_summary (1).csv')
```

```

# Convert decimal values to percentages
for col in ['accuracy', 'precision', 'recall', 'f1']:
    df[col] = df[col] * 100

# Set up the figure and axis
fig, ax = plt.subplots(figsize=(12, 8))
models = df['Model'].tolist()
x = np.arange(len(models))
width = 0.2

#bars for each metric
ax.bar(x - width*1.5, df['accuracy'], width, label='Accuracy', color='#4572C4')
ax.bar(x - width/2, df['precision'], width, label='Precision', color='#ED7D31')
ax.bar(x + width/2, df['recall'], width, label='Recall', color='#A5A5A5')
ax.bar(x + width*1.5, df['f1'], width, label='F1 Score', color='#FFC000')

# Customize the chart
ax.set_title('Model Performance Metrics Comparison (%)', fontsize=16, pad=20)
ax.set_xlabel('Models', fontsize=14, labelpad=10)
ax.set_ylabel('Score (%)', fontsize=14)
ax.set_xticks(x)
ax.set_xticklabels(models, rotation=45, ha='right')
ax.set_ylim(80, 100) # Set y-axis range from 80% to 100%
ax.grid(axis='y', linestyle='--', alpha=0.7)

# Add a Legend
ax.legend(loc='upper center', bbox_to_anchor=(0.5, -0.15),
        fancybox=True, shadow=True, ncol=4)

# Add percentage values on top of the bars
for i, metric in enumerate([df['accuracy'], df['precision'], df['recall'], df['f1']]):
    for j, value in enumerate(metric):
        position = x[j] - width*1.5 + width*i
        ax.text(position, value + 0.5, f'{value:.1f}%',
                ha='left', va='bottom', rotation=90, fontsize=8)

# Adjust to make room for the Legend
plt.tight_layout()
plt.subplots_adjust(bottom=0.25)

# Save the figure as an image file
plt.savefig('model_metrics_comparison_percent.png', dpi=300, bbox_inches='tight')
plt.show()

```